Limitations of Automated Binary Vulnerability Analysis in Optimized and Stripped Binaries

Los Escalante
losesc@vt.edu
Milestone Report

1 Revised scope and thesis

The proposal asks whether automated static analysis and LLM based vulnerability detectors can stay reliable once binaries are compiled with more aggressive optimization and stripped of symbols. Three changes were made following instructor feedback and a closer read of the literature.

First, the baseline changed. The proposal named `cppcheck`, but `cppcheck` analyzes C source and never inspects a compiled binary, so it can't measure any effect of optimization or stripping. It is kept only as a source level reference point. The binary-level static baseline is now `cwe_checker`, which operates on the lifted binary and covers the three target weakness classes. Second, the targets are reproduction grade: each program has a vulnerable and a fixed (patched) version, so false positives and hallucination are measured, not only missed bugs. Third, optimization and symbol stripping now vary independently in a 2×2 design, as the feedback requested, instead of one `-O3 -s` build.

Thesis: We reproduce a published source level vulnerability detection result and test how far it degrades when the same programs are presented as decompiled binaries, isolating the effect of compiler optimization (`-O0` vs. `-O3`) from the effect of symbol stripping, in order to mark the point at which automated triage is no longer reliable and manual reverse engineering analysis is mandatory.

2 Reproduction target and baseline

We reproduce a specific result from Khare et al. [1], who evaluated 16 LLMs on five vulnerability datasets. We adopt their *Juliet C/C++* setting and their exact prompts (system prompt, Basic, and CWE specific) at temperature 0. The claim we reproduce is their reported Juliet C/C++ detection performance, summarized in Table 1. Their best model on this dataset (Qwen-2.5-14B, CWE-specific prompt) reaches accuracy 0.65 and F1 0.72, while the CodeQL static baseline reaches accuracy 0.72 but recall only 0.44. A notable pattern is that most models achieve high recall ($\sim$0.9–1.0) with precision near 0.5: they call almost everything vulnerable. Our fixed-version programs are designed to expose exactly this bias and to track whether it worsens on binaries. Our eight targets follow the Juliet style but are hand-written for this milestone; a Juliet C/C++ subset is integrated (`fetch_juliet.sh`) and will replace them for the final report.

Source (Juliet C/C++)	Method	Acc	Prec	Rec	F1
Khare et al. [1]	CodeQL (static)	0.72	0.99	0.44	0.61
Khare et al. [1]	Qwen-2.5-14B (CWE)	0.65	0.60	0.89	0.72
Khare et al. [1]	GPT-4 (CWE)	0.58	0.54	0.95	0.69
This work (reproduction)	this work (8 hand-written, source)	0.875	0.875	0.875	0.875

Table 1: Published Juliet C/C++ numbers we reproduce, and our source-level reproduction row (our 8 hand-written targets, source level).

3 What is implemented

The full pipeline runs end to end, only the model inference step requires the local GPU and is run on the author's machine (Ollama, `qwen2.5-coder:14b`, temperature 0). The stages are:

- **Targets.** Eight C programs, each with a vulnerable and a fixed variant (16 source cases), covering buffer overflow (CWE-121/787), format string (CWE-134), and use-after-free / double-free (CWE-416/415). Juliet-style identifiers are renamed to neutral names so the model cannot read the answer from a symbol like `CWE121_..._bad`.
- **Build matrix.** `compile.sh` produces the 2×2 builds per variant. Stripping is applied to a *copy* of the compiled binary (`strip --strip-all`), not via recompilation, so the machine code is byte-identical to its symbol-bearing twin. `_FORTIFY_SOURCE` and the stack protector are disabled so `-O3` does not replace `strcpy` with `__strcpy_chk` or insert canaries, which would change the bug rather than merely optimize around it.
- **Build verification.** `verify_builds.sh` confirms (1) each stripped binary has a `.text` section byte identical to its kept twin, so symbols are the only difference, and (2) every vulnerable binary still contains its bug after compilation - memory bugs confirmed by AddressSanitizer, format strings confirmed by a crash under a format-specifier payload - at both `-O0` and `-O3`. All 16 vulnerable builds are confirmed.
- **Decompilation.** `decompile.sh` exports Ghidra headless pseudo C per binary (`objdump` fallback without Ghidra), plus a comment-stripped source corpus for the source condition.
- **Inference + scoring.** `run_pipeline.py` applies the Khare prompts and logs a structured verdict per item; `score.py` computes accuracy, precision, recall, F1, CWE match, and hallucination per condition, and the factor-isolation deltas.

4 Evaluation design and the development/evaluation split

The detection task is binary (vulnerable vs. not), scored against the known label of each variant. Measures: accuracy, precision, recall, and F1 for detection; *CWE-match* (of correctly flagged vulnerable cases, the fraction assigned the right CWE); and *hallucination* (fraction of fixed programs flagged vulnerable). The five conditions are `source`, and the four binary builds `O0/kept`, `O0/stripped`, `O3/kept`, `O3/stripped`. Because the project reproduces and diagnoses rather than trains, the "split" is a prompt-development vs. evaluation split: prompts and parsing were fixed on two held-out programs (one overflow, one UAF) and never changed based on the remaining evaluation targets. Factor isolation compares conditions that differ in exactly one variable: `O0/kept` vs. `O3/kept` isolates optimization; `O0/kept` vs. `O0/stripped` isolates stripping.

5 Preliminary results and interpretation

The pipeline, baseline reproduction, and verification are complete. Table 2 reports detection and triage metrics per condition from the model run (`qwen2.5-coder:14b`, temperature 0).

Condition	Acc	Prec	Rec	F1	CWE-match	Halluc.
source	0.875	0.875	0.875	0.875	1.000	0.125
O0 / kept	0.625	0.667	0.500	0.571	1.000	0.250
O0 / stripped	0.562	0.667	0.250	0.364	1.000	0.125
O3 / kept	0.600	0.600	0.429	0.500	1.000	0.250
O3 / stripped	0.600	0.600	0.429	0.500	1.000	0.250

Table 2: Detection and triage metrics per condition (16 cases each).

Findings. Detection degrades sharply from source to binary: F1 falls from 0.875 on source code to 0.571 on the symbol-bearing `-O0` binary (a 0.30 drop) before any optimization or stripping - presenting compiled code instead of source is the single largest effect. Among the binary factors, symbol stripping hurts more than optimization: at `-O0`, stripping costs 0.207 F1 (0.571 to 0.364),

while -O0 to -O3 with symbols kept costs only 0.071. The failure mode is missed detections, not false alarms - recall collapses from 0.875 (source) to 0.25 (-O0/stripped) while precision stays near 0.6–0.67, so the model goes blind rather than noisy. When it flags a bug it names the correct CWE (CWE-match 1.0 across all conditions), so the breakdown is in detection, not classification.

Class-dependent degradation. The aggregate hides a strong per-class split. Use-after-free and double-free (CWE-416/415) are still detected on -O3-stripped binaries, because the `malloc`/`free` call pattern survives compilation as recognizable PLT calls. Buffer overflows and format-string bugs (CWE-121/134) are missed on nearly all binary builds, because the vulnerable operation (`strcpy`, `printf(user)`) compiles away into generic register and memory instructions with no library-call signature. Automated binary triage is therefore reliable only for the library-call-based bug classes in this set.

Instruction breakdown at `-O3`. For 2 of 16 -O3 cases the model abandoned the verdict format and instead narrated the disassembly, once misidentifying the target as glibc's `fwrite`. These are logged as unparseable and excluded from metrics. We treat them as a triage failure in their own right: at high optimization the input is far enough out of distribution that the model stops performing the task, which supports routing -O3-stripped binaries to manual Ghidra/GDB analysis.

6 What will be studied next (the distribution shift)

The stress condition is the structural distribution shift from source to -O3-stripped binary: identical semantics, radically different surface form. The remaining work is: (1) run `cwe_checker` on all four binary builds as the non-LLM binary baseline; (2) define the triage decision rule concretely - an alert is trusted only above a recall threshold with LLM and `cwe_checker` agreement, else it is routed to manual Ghidra/GDB - and report how often each condition would be routed to manual analysis; (3) a short qualitative failure analysis mapping specific compiler transformations (inlining, `strcpy` vectorization, dead-store elimination) to specific triage errors.

7 Completion plan

- **By Fri (milestone):** pipeline + baseline reproduction + verification done (complete); source and binary runs logged; Table 2 filled.
- **Final report:** add `cwe_checker` baseline; triage decision rule and routing rates; failure analysis; expand targets if time allows.
- **Risks:** a bug optimized away at -O3 (mitigated by the ASan survival check, all 16 currently pass); Ghidra headless setup (mitigated by the `objdump` fallback, already working).

Class-survey entry

This project studies binary vulnerability triage. It tests whether Khare et al.'s source-level LLM detection on Juliet C/C++ holds on optimized and/or stripped decompiled binaries, using a 2×2 design (-O0/-O3 $\times$ symbols kept/stripped) that compares the same LLM and prompts to the source baseline, with `cwe_checker` planned as a non-LLM baseline. The goal is to find where automated triage fails and manual Ghidra/GDB analysis becomes necessary.

Domain	Task	Baseline	Distribution shift studied	Primary metric	Finding so far
Binary vuln. triage	Bug detection + CWE ID (Juliet C/C++)	Khare et al. source LLM (Qwen-2.5-14B)	Source $\rightarrow$ binary: -O0/-O3 $\times$ kept/stripped	F1, CWE match	Source F1 0.88 $\rightarrow$ 0.36 stripped; stripping $>$ optimization

Contributions. Solo project. Los Escalante: target curation, build/verification scripts, decompilation pipeline, prompt pipeline and scorer, the Khare et al. reproduction, and this report. **Artifacts.** Code and data: `github.com/losesc/binvuln`. Scripts: `compile.sh`, `verify_builds.sh`, `decompile.sh`, `run_pipeline.py`, `score.py`.

References

[1] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, M. Naik. "Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities." arXiv:2311.16169, 2024 (ICST 2025).

[2] T. Boland, P. E. Black. "Juliet 1.1 C/C++ and Java Test Suite." *IEEE Computer*, 2012. NIST SAMATE SARD.

[3] Fraunhofer FKIE. "cwe_checker." `https://github.com/fkie-cad/cwe_checker`.